

Day 3 Capstone (SCRAMBLED): Form → Sheet → Analyze → AI

DICT Data Science Seminar — Baguio

⚠ **The cells below are OUT OF ORDER on purpose.**

Your group's challenge: **figure out the correct sequence and rearrange the cells** so the whole pipeline runs top to bottom without errors.

How to move a cell in Colab: click the cell, then use the ↑ / ↓ **move buttons** in the little toolbar at its top-right corner (or the cell menu).

How to figure out the order — think about what must happen first:

- You can't *use* a library before you **import** it, and you can't import a package before you **install** it.
- You can't open a Google Sheet before you **sign in** to Google.
- You can't analyze or chart data before you've **loaded** it.
- You can't *predict* with a model before you've **uploaded and loaded** it.
- You can't test **confidence** before you've **classified** the images.

Each cell has a letter (A–J) and a one-line hint of what it does. When you think the order is right, **run them top to bottom** — if it works with no errors, you solved it!

(Tip: write your guessed order on paper first, e.g. D → H → F → ... , then move the cells to match.)

Cell A

Draws charts of the form responses.

CODE — COPY THIS INTO A CELL

```
# ===== VISUALIZE THE FORM DATA =====  
# Bar charts for short text/choice answers; histograms for numbers.
```

```

skip = {IMAGE_COL, 'Timestamp', 'Email Address', 'Email address'}
cat_cols = [c for c in df.columns
             if c not in skip and df[c].dtype == object and 1 < df[c].nunique() <= 15]
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()

plots = cat_cols[:3] + num_cols[:3]
if plots:
    n = len(plots); cols = 3; rows = (n + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(6*cols, 4.2*rows))
    axes = np.array(axes).flatten()
    for i, c in enumerate(plots):
        if c in num_cols:
            axes[i].hist(df[c].dropna(), bins=10, color='steelblue', edgecolor='black',
alpha=0.8)
        else:
            vc = df[c].value_counts()
            axes[i].bar(vc.index.astype(str), vc.values, color='teal', alpha=0.85)
            axes[i].tick_params(axis='x', rotation=45)
            axes[i].set_title(str(c)[:40], fontweight='bold')
    for j in range(len(plots), len(axes)):
        axes[j].axis('off')
    plt.tight_layout(); plt.show()
else:
    print("No chartable columns detected yet – collect a few more responses.")

print("\nINTERPRETATION (write 2-3 sentences): what stands out in the responses above?")

```

Cell B

Prints data types, a numeric summary, and missing values.

CODE — COPY THIS INTO A CELL

```

# ===== EXPLORE & DESCRIBE THE DATA =====
print("Data types:")
print(df.dtypes)

num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
if num_cols:
    print("\nNumeric summary:")
    display(df[num_cols].describe().round(2))

print("\nMissing values per column:")
miss = df.isnull().sum()
print(miss[miss > 0] if miss.sum() else " None")

```

Cell C

Downloads each uploaded image and predicts its class.

CODE — COPY THIS INTO A CELL

```
# ===== CLASSIFY EACH UPLOADED DOCUMENT + CONFIDENCE =====
def extract_drive_id(url):
    m = re.search(r'[-\w]{25,}', str(url))
    return m.group(0) if m else None

def download_drive_file(file_id, dest):
    req = drive_service.files().get_media(fileId=file_id)
    fh = io.FileIO(dest, 'wb')
    dl = MediaIoBaseDownload(fh, req); done = False
    while not done:
        _, done = dl.next_chunk()
    fh.close(); return dest

os.makedirs('/content/form_images', exist_ok=True)
preds, confs = [], []
for i, row in df.iterrows():
    fid = extract_drive_id(row.get(IMAGE_COL, ''))
    if not fid:
        preds.append(None); confs.append(None); continue
    try:
        dest = download_drive_file(fid, f'/content/form_images/row{i}.jpg')
        label, conf = predict_image(dest)
        preds.append(label); confs.append(conf)
    except Exception as e:
        print(f"Row {i}: {e}"); preds.append(None); confs.append(None)

df['AI_Prediction'] = preds
df['AI_Confidence'] = confs
done = df.dropna(subset=['AI_Confidence'])
print(f"Classified {len(done)} of {len(df)} uploaded documents.")
display(df[[IMAGE_COL, 'AI_Prediction', 'AI_Confidence']].head(20))
```

Cell D

Installs the Python packages.

CODE — COPY THIS INTO A CELL

```
!pip install gspread -q
```

Cell E

Loads the model and defines the predict() function.

CODE — COPY THIS INTO A CELL

```
# ===== LOAD THE MODEL + PREDICTION FUNCTION =====
import tensorflow as tf
```

```

if savedmodel:
    model = tf.saved_model.load(os.path.dirname(savedmodel[0]))
    predict_fn = model.signatures['serving_default']; MODE = 'savedmodel'
elif keras_files:
    model = tf.keras.models.load_model(keras_files[0], compile=False)
    predict_fn = None; MODE = 'keras'
else:
    raise SystemExit("No model found in the zip.")

class_names = []
if labels_path:
    with open(labels_path) as f:
        for line in f:
            t = line.strip()
            if t:
                p = t.split(' ', 1)
                class_names.append(p[1] if len(p) == 2 and p[0].isdigit() else t)
print(f"Model loaded ({MODE}). Classes:", class_names)

def predict_image(path):
    img = Image.open(path)
    if img.mode != 'RGB':
        img = img.convert('RGB')
    arr = np.asarray(img.resize((224, 224))).astype(np.float32)
    x = (arr / 255.0)[None, ...] if MODE == 'savedmodel' else ((arr / 127.5) - 1.0)[None, ...]
    if MODE == 'savedmodel':
        out = predict_fn(tf.constant(x)); probs = out[list(out.keys())[0]].numpy()[0]
    else:
        probs = model.predict(x, verbose=0)[0]
    idx = int(np.argmax(probs))
    label = class_names[idx] if idx < len(class_names) else f'Class {idx}'
    return label, float(probs[idx])

```

Cell F

Signs in to Google (Sheets + Drive).

CODE — COPY THIS INTO A CELL

```

# ===== CONNECT TO GOOGLE (Sheet + Drive) =====
# A popup asks you to choose your Google account (the one that OWNS the form/sheet) and Allow.
auth.authenticate_user()
creds, _ = default()
gc = gspread.authorize(creds)
drive_service = build('drive', 'v3')
print("Connected to Google Sheets + Drive.")

```

Cell G

Charts the model's confidence and flags low ones for review.

CODE — COPY THIS INTO A CELL

```
# ===== TEST THE MODEL'S CONFIDENCE =====
if len(done) == 0:
    print("No images classified yet – check SHEET_ID, IMAGE_COL, and that responses have uploads.")
else:
    fig, axes = plt.subplots(1, 2, figsize=(13, 5))
    axes[0].hist(done['AI_Confidence'], bins=10, color='gold', edgecolor='black')
    axes[0].axvline(done['AI_Confidence'].mean(), color='red', linestyle='--',
                    label=f"Mean {done['AI_Confidence'].mean():.1%}")
    axes[0].axvline(0.8, color='orange', linestyle=':', label='80% review line')
    axes[0].set_title('Model Confidence Distribution'); axes[0].set_xlabel('Confidence');
    axes[0].legend()

    by = done.groupby('AI_Prediction')['AI_Confidence'].mean().sort_values()
    axes[1].barh(by.index, by.values, color='teal')
    axes[1].set_xlim(0, 1); axes[1].set_title('Average Confidence by Predicted Class')
    plt.tight_layout(); plt.show()

    print("CONFIDENCE REPORT")
    print(f"  Average confidence: {done['AI_Confidence'].mean():.1%}")
    print(f"  High (>=90%): {(done['AI_Confidence'] >= 0.9).sum()} | "
          f"Low (<80%, review): {(done['AI_Confidence'] < 0.8).sum()} of {len(done)}")
    print("  Predictions per class:")
    for cls, n in done['AI_Prediction'].value_counts().items():
        print(f"    {cls}: {n}")
    print("\nINTERPRETATION: Is the model confident? Which uploads need a human to review (<80%)?")
```

Cell H

Imports the libraries (pandas, gspread, tensorflow, ...).

CODE — COPY THIS INTO A CELL

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os, io, re
import gspread
from google.colab import auth
from google.auth import default
from googleapiclient.discovery import build
from googleapiclient.http import MediaIoBaseDownload
from IPython.display import display
print("Libraries loaded.")
```

Cell I

Opens your Google Sheet into a table (set SHEET_ID + IMAGE_COL).

CODE — COPY THIS INTO A CELL

```
# ===== LOAD YOUR FORM RESPONSES =====
# Paste your linked response Sheet's ID, and the exact column title of the IMAGE UPLOAD
question.
SHEET_ID = 'PASTE_YOUR_SHEET_ID_HERE'
IMAGE_COL = 'Upload your document' # change to your form's file-upload question title

ws = gc.open_by_key(SHEET_ID).get_worksheet(0)
df = pd.DataFrame(ws.get_all_records())
print(f"Loaded {len(df)} responses with {df.shape[1]} columns.")
print("Columns:", list(df.columns))
display(df.head())
```

Cell J

Uploads and unzips your Teachable Machine model .zip.

CODE — COPY THIS INTO A CELL

```
# ===== UPLOAD YOUR TEACHABLE MACHINE MODEL (.zip) =====
# Export Model -> Tensorflow -> "Download my model" gives a .zip. Upload it here; it auto-
arranges.
import zipfile, glob, shutil
from PIL import Image
from google.colab import files

print("Choose your Teachable Machine model .zip ...")
uploaded = files.upload()
zip_names = [n for n in uploaded if n.lower().endswith('.zip')]
if not zip_names:
    raise SystemExit("No .zip found. Re-run and select your model zip.")

MODEL_ROOT = '/content/model'
if os.path.exists(MODEL_ROOT):
    shutil.rmtree(MODEL_ROOT)
os.makedirs(MODEL_ROOT, exist_ok=True)
with zipfile.ZipFile(zip_names[0]) as z:
    z.extractall(MODEL_ROOT)

savedmodel = glob.glob(f'{MODEL_ROOT}/**/*.saved_model.pb', recursive=True)
keras_files = glob.glob(f'{MODEL_ROOT}/**/*.h5', recursive=True)
label_files = glob.glob(f'{MODEL_ROOT}/**/*.labels.txt', recursive=True)
labels_path = label_files[0] if label_files else None
```

```
print("Found -> SavedModel:", bool(savedmodel), "| Keras:", bool(keras_files), "| labels:",  
      bool(labels_path))
```